

# Project Files Structure 001 - SED

Ordoñez Silva, Yonel Jr.

## Indice

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>I. Propuesta de Estructura de Archivos del Proyecto</b> | <b>1</b> |
| 1.1. Estructura General del Proyecto . . . . .             | 1        |
| 1.2. Propuestas de Mejora de Estructura . . . . .          | 3        |

## I. Propuesta de Estructura de Archivos del Proyecto

### 1.1. Estructura General del Proyecto

El flujo de cambios y versiones se hara a traves de `git flow` en un unico repositorio

Si se desea interactuar con otro modulo se tiene que realizar una fusion (`merge`) hacia la rama `develop` de la rama donde se encuentra desarrollando el modulo solicitante con la rama del modulo objetivo.

Para coordinar los componentes fuera del dominio (`src/main/` o `src/main/java/com/zentry/sed`) se realizaran reuniones generales donde se discutira y observara el avance de cada integrante, con el objetivo de poder realizar la orquestacion y configuracion necesarias de acuerdo a la configuracion inividual de cada modulo desarrollado por cada integrante.

La estrucutra propuesta para una arquitectura Monolitica + Capas combinado con los patrones de diseño MVC + Clean Architecture es la siguiente:

```
app/  
  src/  
    main/  
      java/  
        com/zentry/sed/  
          config/      # Configuración de Spring, Beans, Seguridad, CORS, etc.  
          utils/       # Utilidades genéricas (fechas, strings, etc.)
```

|                        |                                                        |
|------------------------|--------------------------------------------------------|
| presentation/          | # Capa de presentación (UI + Controladores)            |
| views/                 | # UI (HTML)                                            |
| controllers/           | # @Controller comunicacion con frontend                |
| models/                | # DTOs y ViewModels que la vista necesita              |
| services/              | # Capa de aplicación (Casos de uso)                    |
| usecases/              | # Casos de uso concretos                               |
| interfaces/            | # Interfaces para los servicios del dominio            |
| core/                  | # Capa de dominio                                      |
| entities/              | # Entidades de dominio                                 |
| value-objects/         | # Objetos de valor (si aplica)                         |
| repositories/          | # Interfaces de los repositorios                       |
| services/              | # Servicios de dominio puros                           |
| infrastructure/        | # Capa de infraestructura                              |
| repositories/          | # Implementaciones de los repositorios                 |
| database/              | # Configuración y acceso a base de datos               |
| entities/              | # Entidades JPA (@Entity)                              |
| mappers/               | # Mappers entre dominio infraestructura                |
| embedded/              | # Opcional, clases embebidas para Id's de tablas       |
| external-apis/         | # Clientes de APIs externas                            |
| config/                | # Configuración de infraestructura (env, logger, etc)  |
| resources/             |                                                        |
| application.properties | # Configuración principal (por defecto)                |
| application.yml        | # O alternativa en formato YAML                        |
| static/                | # Archivos estáticos como CSS, JS, imágenes            |
| style.css              |                                                        |
| app.js                 |                                                        |
| templates/             | # Plantillas HTML (Thymeleaf, Freemarker, etc.)        |
| index.html             |                                                        |
| i18n/                  | # Archivos de internacionalización (mensajes por       |
| messages_en.properties |                                                        |
| messages_es.properties |                                                        |
| db/                    |                                                        |
| migration/             | # Scripts SQL para migraciones (ej: Flyway, Liquibase) |
| V1__init.sql           |                                                        |
| seed/                  | # Datos de prueba para poblar la DB                    |
| logback-spring.xml     | # Configuración del sistema de logs (Logback)          |
| banner.txt             | # Banner que se muestra al arrancar la app             |
| META-INF/              | # Archivos especiales como manifest, spring.factories  |
| test/                  |                                                        |

```
java/  
    com/miempresa/myapplication/  
        presentation/  
        services/  
        core/  
        infrastructure/  
        ...
```

## 1.2. Propuestas de Mejora de Estructura

1. Se podría modularizar las carpetas del backend y frontend para contar con vistas y lógica separadas por módulos de requisitos.